



Trabalho Prático — Teste Funcional

Prof. Iaçanã Ianiski Weber

Entrega: 12/04 (23:59) - no Moodle

Contexto: Calculadora de Dano de RPG

Em jogos de RPG, o cálculo de dano em combate envolve diversas variáveis: o poder de ataque do personagem, a defesa do alvo, o tipo de ataque utilizado e a possibilidade de um golpe crítico. Neste trabalho, você testará uma função que implementa essa mecânica de combate, aplicando técnicas de **teste funcional** (caixa preta) para projetar casos de teste e detectar defeitos em versões incorretas da implementação.

Especificação da Função

A função `calcular_dano` recebe quatro parâmetros inteiros e retorna um inteiro representando o dano causado:

```
int calcular_dano(int poder_ataque, int defesa, int tipo_ataque, int eh_critico);
```

Parâmetros

Parâmetro	Tipo	Domínio	Descrição
poder_ataque	int	[1, 150]	Poder de ataque do personagem
defesa	int	[0, 150]	Defesa do alvo
tipo_ataque	int	{1, 2, 3}	1 = físico, 2 = mágico, 3 = verdadeiro
eh_critico	int	{0, 1}	0 = normal, 1 = golpe crítico

Regras de Cálculo

1. **Validação:** Se qualquer parâmetro estiver fora do domínio, retornar -1.
2. **Dano base** conforme o tipo de ataque:
 - **Físico** (tipo 1): $\text{dano} = \text{poder_ataque} - \text{defesa}$
 - **Mágico** (tipo 2): $\text{dano} = \text{poder_ataque} - \lfloor \text{defesa}/2 \rfloor$ (divisão inteira)
 - **Verdadeiro** (tipo 3): $\text{dano} = \text{poder_ataque}$ (ignora a defesa)
3. **Dano mínimo:** Se $\text{dano} < 1$, então $\text{dano} = 1$.
4. **Golpe crítico:** Se $\text{eh_critico} = 1$, então $\text{dano} = \text{dano} \times 2$.
5. **Retorno:** O valor final de dano.

Nota: As regras são aplicadas na ordem acima — primeiro calcula-se o dano base, depois aplica-se o mínimo, e por fim o multiplicador crítico.

Tarefas

1. **Particionamento de Equivalência:**

a) Identifique todas as **classes de equivalência inválidas** (entradas fora do domínio).

- b) Identifique todas as **classes de equivalência válidas** (combinações de tipo de ataque e golpe crítico, incluindo subclasses onde o dano mínimo é ativado).
- c) Projete pelo menos um caso de teste por classe.

2. Análise do Valor Limite:

- a) Identifique as **fronteiras** de cada parâmetro.
- b) Identifique a fronteira do **dano mínimo**.
- c) Projete casos de teste para os valores *on* e *off* de cada fronteira.

3. Suíte de testes: Consolide os casos de teste em uma tabela com as colunas: *ID*, *poder_ataque*, *defesa*, *tipo_ataque*, *eh_critico*, *resultado_esperado*, *técnica*, *classe coberta*.

4. Execução contra versões incorretas:

- a) Você receberá **10 versões compiladas** da função como arquivos objeto (*versao01.o* a *versao10.o*), cada uma contendo exatamente **um defeito**. O protótipo da função está no arquivo *calcular_dano.h*.
- b) Implemente seus casos de teste usando o framework **Unity**, disponível em:
<https://github.com/iacanaweber/Unity>
O projeto está na pasta *examples/calcular_dano/*.
A estrutura já está pronta — basta adicionar seus testes no arquivo *test/TestCalcularDano.c*.
- c) Para trocar a versão sob teste, altere a variável **VERSAO** no *makefile* (ou passe via linha de comando):

```
# Testar a versao correta:
make VERSAO=correto.o

# Testar a versao 03:
make VERSAO=versao03.o
```

- d) Para cada versão, identifique quais testes **falharam** e descreva o **defeito provável**.

Estrutura do Projeto Unity

O projeto segue a mesma estrutura dos exemplos vistos em aula:

```
calcular_dano/
src/
  calcular_dano.h      # Prototipo da funcao
  correto.o           # Versao correta (referencia)
  versao01.o ... versao10.o # Versoes com defeitos
test/
  TestCalcularDano.c   # Seus casos de teste aqui
test_runners/
  TestCalcularDano_Runner.c # Registrar cada teste
  all_tests.c          # Ponto de entrada
makefile
```

Como adicionar testes

Em `test/TestCalcularDano.c`, adicione um teste por caso:

```
TEST(CalcularDano, DanoFisicoNormal)
{
    TEST_ASSERT_EQUAL(70, calcular_dano(100, 30, 1, 0));
}
```

Em `test/test_runners/TestCalcularDano_Runner.c`, registre cada teste:

```
TEST_GROUP_RUNNER(CalcularDano)
{
    RUN_TEST_CASE(CalcularDano, DanoFisicoNormal);
    /* ... demais testes ... */
}
```

Compile e execute com `make` ou `make VERSAO=versaoNN.o`.

Entregáveis

- Tabela de classes de equivalência (válidas e inválidas)
- Tabela de fronteiras e valores limite
- Suíte de testes completa (tabela)
- Código-fonte dos testes Unity (`TestCalcularDano.c` e `TestCalcularDano_Runner.c`)
- Relatório de execução: para cada versão, quais testes falharam e qual o defeito identificado